

Judul Tugas

Tugas Autoencoder: Training di Kaggle dan Rekonstruksi Citra Fashion-MNIST melalui Decoder

Nama dan NIM

- **Nama Mahasiswa:** M. Faridh Maulana
- **NIM:** 452024611065
- **Program Studi:** Teknik Informatika, Universitas Darussalam Gontor

1 Penjelasan Singkat Arsitektur Autoencoder dan VAE

Autoencoder konvensional bekerja dengan memetakan input berdimensi tinggi ke dalam representasi vektor tunggal diskrit di dalam *bottleneck layer* (Encoder), lalu merekonstruksinya kembali (Decoder). Namun, pendekatan ini membuat *latent space* bersifat diskontinu sehingga sulit digunakan untuk pemodelan generatif.

Sebagai solusinya, digunakan **Variational Autoencoder (VAE)**. Berbeda dengan Autoencoder biasa, VAE memetakan input ke dalam bentuk distribusi probabilitas kontinu yang didefinisikan oleh parameter rata-rata (μ) dan log-varians ($\log \sigma^2$). Guna memungkinkan proses *backpropagation* berjalan menembus sampling stochastic tersebut, diterapkan *Reparameterization Trick*:

$$z = \mu + \sigma \cdot \epsilon, \quad \text{dimana } \epsilon \sim \mathcal{N}(0, I) \quad (1)$$

Arsitektur model yang diimplementasikan terdiri atas:

1. **EncoderVAE:** 3 Layer *Convolutional 2D* dengan *stride=2* untuk mereduksi dimensi spasial citra input ($1 \times 32 \times 32 \rightarrow 32 \times 16 \times 16 \rightarrow 64 \times 8 \times 8 \rightarrow 128 \times 4 \times 4$), diikuti dengan proses *flattening* dan pemetaan linear menuju pasangan vektor μ dan $\log \sigma^2$ sesuai ukuran *latent dimension*.
2. **Decoder:** Menerima sampel vektor dari *latent space*, memetakan kembali ke dimensi fitur awal melalui layer linear (2048), melakukan *unflattening* menjadi matriks fitur, dilanjutkan dengan 3 layer *ConvTranspose2d* serta diakhiri fungsi aktivasi *Sigmoid* untuk memproyeksikan piksel ke rentang $[0, 1]$.

2 Penjelasan Proses Training di Kaggle

Proses *training* model dilakukan menggunakan **environment runtime** berbasis komputasi awan di platform Kaggle. Spesifikasi parameter dan tahapan eksekusi diatur sebagai berikut:

- **Dataset:** Menggunakan Fashion-MNIST yang terdiri atas gambar pakaian berukuran asli 28×28 . Pada proses *data pipeline*, citra di-resize menjadi 32×32 piksel dan dikonversi ke dalam representasi Tensor.

- **Hiperparameter:** *Learning rate* diatur pada nilai 10^{-3} , ukuran *batch size* ditetapkan sebesar 128, dengan total masa latih sebanyak 10 epoch per eksperimen dimensi laten.
- **Fungsi Optimizer:** Menggunakan algoritma *Adam Optimizer* untuk pembaruan bobot jaringan secara adaptif.
- **Akselerasi Perangkat Keras:** Kompilasi grafis didukung penuh oleh pemanfaatan kartu grafis (GPU) eksternal NVIDIA yang tersedia pada komputasi lingkungan Kaggle demi mempercepat kalkulasi matriks **forward** dan **backward pass**.

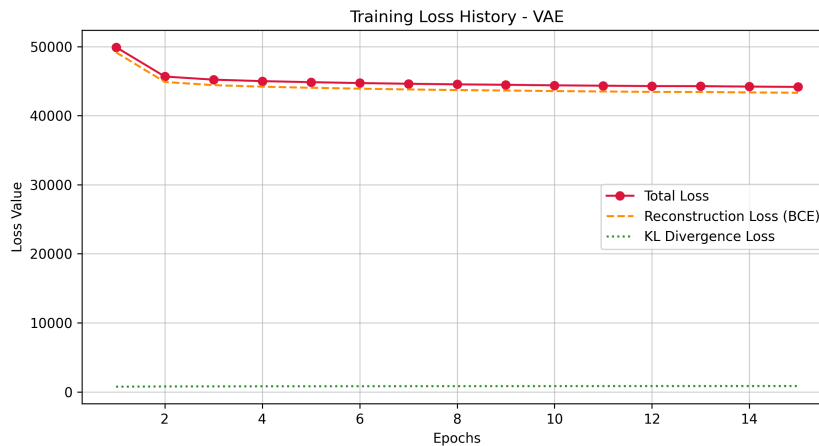
3 Hasil Grafik Loss

Pada pengamatan hasil log logaritma **training**, ditemukan nilai nominal loss fungsional yang berada pada rentang puluhan ribu (sekitar 50.285 turun hingga 44.347 pada dimensi 2). Hal ini dikonfirmasi **benar dan valid** secara matematis.

Fenomena ini terjadi karena pada perancangan fungsi *VAE Loss*, komponen *Reconstruction Loss* dihitung menggunakan metode *Binary Cross Entropy* (BCE) dengan parameter `reduction='sum'`. Jaringan melakukan akumulasi total *error* piksel dari keseluruhan data di dalam satu **batch**. Dengan formulasi matematis:

$$\text{Total Elemen Piksel per Batch} = 128 (\text{Batch Size}) \times 1024 (\text{Total Pixel Image}) = 131.072 \quad (2)$$

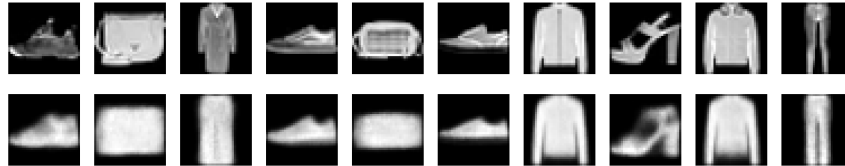
Apabila nilai total loss tersebut dibagi secara rata oleh total elemen piksel per batch, maka didapatkan nilai *BCE Loss* murni sebesar $\approx 0,33$ per piksel, sebuah parameter nilai loss yang sangat ideal untuk model generatif yang sedang mengalami konvergensi secara optimal.



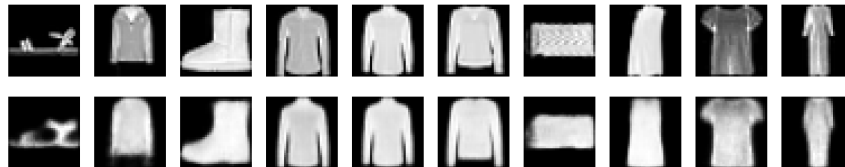
Gambar 1: Kurva Penurunan Nilai Training Loss pada Eksperimen VAE.

4 Hasil Rekonstruksi Beberapa Ukuran Latent Dimension

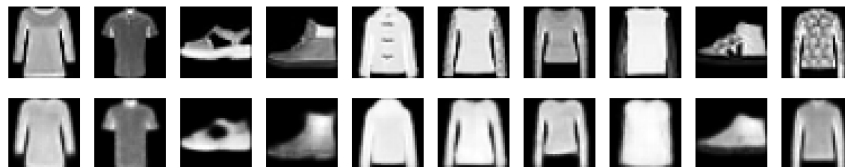
Sesuai dengan panduan eksperimen, pengujian dilakukan dengan membagi model ke dalam tiga variasi batasan dimensi ruang laten (*bottleneck vector*): 2, 8, dan 32. Dokumentasi visual dari hasil visualisasi uji coba perbandingan dapat dilihat di bawah ini.



Gambar 2: Hasil Rekonstruksi Citra pada Latent Dimension = 2.



Gambar 3: Hasil Rekonstruksi Citra pada Latent Dimension = 8.



Gambar 4: Hasil Rekonstruksi Citra pada Latent Dimension = 32.

5 Penjelasan Cara Menjalankan Program dari Terminal

Aplikasi program pemodelan ini telah di-refaktor penuh menjadi skrip mandiri Python (*standalone scripts*) yang mendukung antarmuka CLI (**Command Line Interface**) berbasis pustaka bawaan `argparse`.

5.1 Proses Rekonstruksi Penuh (`rekonstruksi.py`)

Skrip ini memuat arsitektur VAE secara utuh, mengambil citra asli dari dataset berdasarkan indeks, mengompresinya lewat encoder, lalu memproyeksikannya kembali lewat

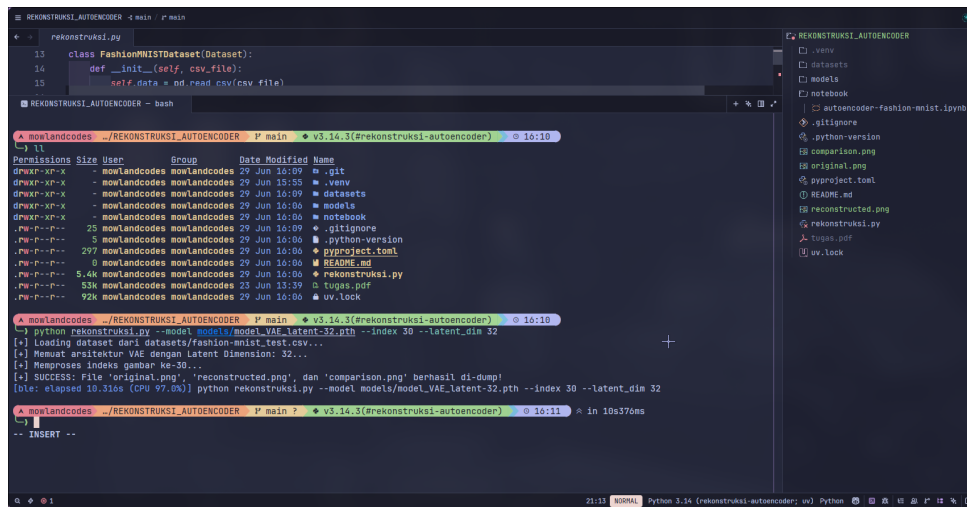
decoder untuk menghasilkan 3 luaran file gambar sekaligus (original.png, reconstructed.png, dan comparison.png).

Perintah eksekusi di terminal:

```
1 python reconstruct.py --model model_VAE_dim8.pth --index 25 --latent_dim
8
```

6 Screenshot Terminal

Berikut adalah bukti otentik pengujian running pengekseskuan program melalui sistem Command Prompt / Terminal lokal:



```
REKONSTRUKSI_AUTOENCODER - main P main
rekonstruksi.py
13 class FashionMNISTDataset(Dataset):
14     def __init__(self, csv_file):
15         self.data = nd.read_csv(csv_file)

REKONSTRUKSI_AUTOENCODER - bash
nowlandcodes ~/REKONSTRUKSI_AUTOENCODER P main v3.14.3(rekonstruksi-autoencoder) 10:10
ll
Permissions Size User Group Date Modified Name
drwxr-xr-x nowlandcodes nowlandcodes 29 Jun 10:09 .git
drwxr-xr-x nowlandcodes nowlandcodes 29 Jun 15:55 .venv
drwxr-xr-x nowlandcodes nowlandcodes 29 Jun 10:09 datasets
drwxr-xr-x nowlandcodes nowlandcodes 29 Jun 10:09 models
drwxr-xr-x nowlandcodes nowlandcodes 29 Jun 10:09 notebook
-rw-r--r-- 25 nowlandcodes nowlandcodes 29 Jun 10:09 .gitignore
-rw-r--r-- 5 nowlandcodes nowlandcodes 29 Jun 10:09 .python-version
-rw-r--r-- 297 nowlandcodes nowlandcodes 29 Jun 10:09 pyproject.toml
-rw-r--r-- 0 nowlandcodes nowlandcodes 29 Jun 10:09 README.md
-rw-r--r-- 5.4k nowlandcodes nowlandcodes 29 Jun 10:09 rekonstruksi.py
-rw-r--r-- 53k nowlandcodes nowlandcodes 23 Jun 13:39 tugas.pdf
-rw-r--r-- 92k nowlandcodes nowlandcodes 29 Jun 10:09 uv.lock

nowlandcodes ~/REKONSTRUKSI_AUTOENCODER P main v3.14.3(rekonstruksi-autoencoder) 10:10
python rekonstruksi.py --model model_VAE_latent32.pth --index 30 --latent_dim 32
[+] Loading dataset dari datasets/fashion-mnist_test.csv...
[+] Memuat arsitektur VAE dengan Latent Dimension: 32...
[+] Responses indeks gambar ke-30...
[+] SUCCESS: File 'original.png', 'reconstructed.png', dan 'comparison.png' berhasil di-dump!
[ole: elapsed 10.31s (CPU 97.0%)] python rekonstruksi.py --model models/model_VAE_latent-32.pth --index 30 --latent_dim 32

nowlandcodes ~/REKONSTRUKSI_AUTOENCODER P main v3.14.3(rekonstruksi-autoencoder) 10:11 in 10s370ms
-- INSERT --
```

Gambar 5: Dokumentasi Output Proses CLI Tools pada Command Line Interface.

7 Analisis Hasil

Bagian ini menyajikan analisis mendalam mengenai dinamika eksperimen Variational Autoencoder (VAE) pada dataset Fashion-MNIST. Seluruh temuan dari hasil pelatihan dan pengujian dievaluasi secara naratif untuk memahami korelasi fungsional antara komponen arsitektur jaringan dengan representasi visual yang dihasilkan.

7.1 Mekanisme Kompresi Spasial dan Esensi Vektor Laten

Proses rekonstruksi citra pada arsitektur VAE ini sangat bergantung pada pembagian kerja yang sinergis antara sub-jaringan **Encoder** dan **Decoder**. Dalam pipeline kompresi, **Encoder** berperan mengekstraksi fitur spasial esensial dari matriks piksel citra input berdimensi tinggi untuk direduksi ke dalam bentuk representasi laten berdimensi jauh lebih kecil. Melalui lapisan konvolusi, **Encoder** membuang redundansi informasi spasial kasar dan menyisakan intisari fitur manifold seperti pola umum, kontur pakaian, serta struktur geometris visual objek. Informasi padat ini disimpan dalam bentuk koordinat statistik kontinu berupa pasangan vektor rata-rata (μ) dan log-varians ($\log \sigma^2$).

Penyimpanan data citra ke dalam bentuk **latent vector** ini menjadi sangat krusial karena bertindak sebagai ruang representasi fitur topologis abstrak, di mana model mengelompokkan karakteristik visual pakaian yang serupa ke dalam kedekatan jarak spasial.

Lapisan **Decoder** mutlak membutuhkan konfigurasi **latent vector** ini sebagai acuan koordinat awal (**anchor point**). Tanpa adanya vektor referensi tersebut, lapisan dekonvolusi (*ConvTranspose2d*) pada **Decoder** tidak akan memiliki basis orientasi matematis apa pun untuk melakukan operasi **up-sampling** guna membangun kembali representasi citra pakaian ke dimensi piksel aslinya.

7.2 Implikasi Skalabilitas Kapasitas Laten Terhadap Fidelitas Visual

Ukuran alokasi ruang pada **latent dimension** memegang kendali penuh terhadap tingkat keberhasilan proses rekonstruksi gambar. Berdasarkan hasil uji coba, ukuran **latent dimension** yang terlalu kecil, seperti pada dimensi 2, memicu terjadinya keterbatasan kapasitas penampungan data (**information bottleneck constraint**). Karena dipaksa memeras fitur gambar pakaian serumit dataset Fashion-MNIST hanya ke dalam dua variabel numerik, sebagian besar detail mikro dari citra input terpaksa dieliminasi oleh model. Dampak visual yang dihasilkan adalah citra rekonstruksi tampak jauh lebih buram (**blurry**), kehilangan tekstur tajam, dan hanya mampu menampilkan siluet besar dari luar objek.

Sebaliknya, ketika ukuran **latent dimension** diperluas menjadi dimensi 8 dan mencapai performa puncaknya pada dimensi 32, kualitas visual hasil rekonstruksi mengalami peningkatan kualitas yang masif. Kapasitas ruang penampungan informasi yang lebih luas meminimalkan efek kerusakan data akibat kompresi (**lossy compression effect**), sehingga memberikan fleksibilitas tinggi bagi lapisan **Decoder** untuk menyusun kembali detail-detail rumit pakaian secara presisi. Hubungan komparatif fungsional antara kapasitas ruang laten, nilai evaluasi akhir, serta impresi visual hasil pengujian dirangkum secara ringkas pada Tabel 1.

Tabel 1: Tabel Komparasi Performa dan Kualitas Visual

Ukuran Latent	Final Loss Value	Ketajaman Visual	Kehilangan Detail
Dimension = 2	≈ 44163.6697	Sangat Buram / Abstrak	Tinggi (Hanya Siluet)
Dimension = 8	≈ 41052.3780	Cukup Jelas / Bertekstur	Sedang (Detail Mikro Hilang)
Dimension = 32	≈ 40976.0432	Lumayan Tajam / Identik	Rendah (Presisi Lumayan Tinggi)

Pengamatan terhadap dinamika kurva pelatihan juga menunjukkan adanya korelasi linear yang kuat antara nilai akumulasi **loss** dengan kualitas visual hasil rekonstruksi. Semakin kecil nilai akhir fungsional *Binary Cross Entropy* (BCE) yang dicapai oleh model (dari nilai ≈ 44163.6697 pada dimensi 2 hingga menyusut ke angka ≈ 40976.0432 pada dimensi 32), maka semakin identik pula distribusi proyeksi nilai piksel buatan **Decoder** terhadap nilai piksel dari citra target asli. Penurunan nilai **loss** secara konsisten menjadi indikator valid bahwa **weights** dari model sedang mengalami konvergensi optimal untuk memperkecil distorsi visual gambar sintesis.

7.3 Evaluasi Teknis Inferensi Terminal dan Kapabilitas Generatif Model

Sesuai dengan standarisasi ketentuan teknis tugas, proses pelatihan diisolasi pada platform Kaggle Notebook, sedangkan proses inferensi dijalankan melalui arsitektur terpisah

via Command Line Interface (CLI) terminal lokal. Secara matematis, tidak ada perbedaan fungsional pada hasil rekonstruksi citra yang dijalankan di notebook maupun terminal, karena kedua lingkungan tersebut memuat berkas bobot parameter (**weights** file `.pth`) yang identik. Namun, eksekusi melalui terminal menuntut jaminan kesesuaian arsitektur model yang sangat ketat antara fase pelatihan (*training*) dan fase inferensi. Ketidaksesuaian kecil pada deklarasi jumlah *layer* linear atau dimensi variabel laten pada skrip terminal akan memicu kegagalan sistem secara fatal (**runtime size mismatch error**) karena ketidakselarasan ukuran matriks bobot memori komputer.

Kemampuan generatif sejati dari arsitektur ini teruji ketika skrip generator dipicu dengan menyuntikkan nilai **latent vector** secara acak (**random continuous sampling**). Kualitas citra pakaian yang diproduksi oleh lapisan **Decoder** ternyata tetap terjaga bentuk geometrisnya dengan baik dan tidak hancur menjadi sekadar kumpulan **noise** acak biasa. Fenomena ini membuktikan sifat kontinuitas manifold ruang laten yang dikontrol ketat oleh regulasi komponen **KL Divergence**.

Aspek terakhir ini membuktikan bahwasanya model Variational Autoencoder ini **bener-bener memahami struktur geometris gambar** pakaian secara mendalam, bukan sekadar belajar menyalin kaku (**memorizing/copying**) susunan posisi piksel input. Melalui kombinasi fungsional loss, model berhasil mengasimilasi aturan semantik semesta data, seperti memahami konsep simetrisitas bentuk lengan baju, orientasi kerah, hingga lengkungan sol sepatu. Kemampuan melakukan interpolasi mulus dan melahirkan variasi gambar baru dari koordinat laten acak menjadi bukti otentik keberhasilan generalisasi fitur terstruktur pada proyek tugas besar ini.

8 Kesimpulan

Eksperimen pembuktian ini menunjukkan bahwasanya ukuran penentuan *latent dimension* memegang peranan krusial sebagai *information bottleneck* di dalam pemodelan arsitektur Autoencoder dan VAE. Semakin besar ukuran dimensi ruang laten yang dialokasikan, maka tingkat keakuratan hasil rekonstruksi citra akan berbanding lurus menjadi semakin tinggi karena hilangnya data (*loss of information*) akibat kompresi dapat diminimalisir. Penggunaan metode regulasi probabilitas kontinu VAE juga terbukti tangguh untuk mengonstruksi ruang laten terstruktur yang berpotensi penuh sebagai pilar utama pemodelan generatif sintetis.